Language models

Lecture 1: From next-token prediction to modern LLM systems

Aarne Talman

27 October 2026

LDA-T318 | LLMs and Generative AI for NLP | University of Helsinki

Course overview

Wk	Dates	Tuesday	Thursday	Teacher
1	**27 / 29 Oct**	**Language models**	**Pretraining**	**Aarne**
2	3 / 5 Nov	Post-training	Lab 2: tune vs prompt	Aarne
3	10 / 12 Nov	Inference	Lab 3: serve	Aarne
4	17 / 19 Nov	Evaluation	Lab 4: evaluate	Jussi
5	24 / 26 Nov	RAG	Lab 5: RAG	Dmitry
6	1 / 3 Dec	Agents	Lab 6: agent	Dmitry
7	8 / 10 Dec	Course recap	Special topics (TBC)	All

Reusing a task and model across labs is recommended, not required.

There are no intermediate submissions; the final report and code archive determine the grade.

Today

1. Course logistics *10 min*
2. What is a language model? *15 min*
3. From base model to application *10 min*
4. Tokenization *10 min*
5. Transformers at a high level *20 min*
6. Choosing and reading models *10 min*
7. Roihu and Lab 1 *10 min*

Take away

By the end of today, you should be able to explain how a language model turns text into tokens, uses context to predict the next token, and generates text one token at a time.

Course logistics

The shape of the course

Over seven weeks, we cover the main stages in developing and using language models:

- architecture and pretraining
- post-training
- evaluation
- inference and serving
- retrieval
- agents

Aarne Talman — AMD (weeks 1, 2, 3 and 7)
Jussi Karlgren — AMD (week 4)
Dmitry Kan — Aiven (weeks 5–6)

Every lab runs on CSC Roihu

NVIDIA GH200 nodes.
One H100-class GPU with 96 GiB,
plus a 72-core ARM Grace CPU.

Enough memory for the planned labs,
including bf16 fine-tuning.

An optional way to structure your project

You may use the same task across several labs. This makes it easier to compare prompting, fine-tuning, evaluation, inference and retrieval. You may change models when useful.

Task	Dataset	Metric	Why it works here
Math problems	GSM8K	exact match	automatically checked answers
NLI	XNLI / MNLI	accuracy	multilingual; small label set
Finnish QA	TyDi QA (fi)	F1 / EM	connects naturally to RAG
Finnish NER	Turku NER	entity F1	tests structured output

You may propose another task if it has **a labelled test set** and **an automatic metric**. You may also change models between labs when that makes sense; explain the reason in your report.

Choosing models for the labs

Suggested models (all open-weight under Apache 2.0):

- Qwen3.5 0.8B, 2B or 4B
- Gemma 4 E2B or E4B

For the week-2 comparison, choose a model that:

- has both **base and instruct** versions — week 2 uses both
- fits comfortably on one GH200
- has a licence that permits academic use
- tokenizes the language of your task reasonably well

If you reuse the same task

Week 1: choose the task and evaluation set
Week 2: compare prompting with fine-tuning
Week 3: serve and measure a model
Week 4: evaluate the results carefully
Weeks 5–6: add retrieval and tools
At the end: submit the report and code

This is one option, not a requirement.

Assessment: one final project

The final project accounts for **100% of the course grade**. Submit two files:

- a **PDF report** with one section for each lecture topic, incorporating the related lab work and results
- a **ZIP archive of your code**, including notebooks, scripts, configurations, small result files and a README with instructions for running the main experiments

Do not include model weights, downloaded datasets, caches or virtual environments in the archive.

Submission

Email both files to `aarne.talman@helsinki.fi` by **31 December 2026**.

No intermediate submissions

Weekly lab work contributes material to the final project, but there is no separate weekly submission.

The report and code are assessed together, with emphasis on sound experiments, clear analysis, understanding and reproducibility.

Policies

AI assistants — allowed for code and for writing. End the report with a short statement of which tools you used and for what. Numbers must come from runs you actually did.

Pairs — may share a project and code. One report, with a short statement of who did what.

Compute etiquette

- close idle GPU Jupyter sessions
- debug on `gputest` before `gpumedium`
- never run training on a login node

Licences — only models and datasets whose licences allow academic use. Name them in the report.

Please release idle GPUs

We are all sharing one course project's compute allocation. A forgotten Jupyter session holds a whole GH200 for twelve hours.

What is a language model?

Start from autocomplete

A **language model** reads some text and returns a **probability for every possible next word**. For example, after *"The capital of Finland is"*:

Candidate next word	Probability	
Helsinki	0.82	**the model has learned a fact**
a	0.06	plausible English, wrong continuation
Finland	0.01	grammatical, semantically odd
purple	0.00001	

Every number is positive, and across the whole vocabulary they sum to one. At each generation step, the model internally produces a distribution like this. An API may return only the selected word unless log probabilities are requested.

Take away

Chat, reasoning, agents and retrieval all build on repeated next-word prediction.

From one word to a whole sentence

If the model only scores the *next* word, how does it score a whole sentence? Multiply, one word at a time.

$$P(\textit{The cat sat}) = \underbrace{P(\text{The})}_{\text{no context}} \times \underbrace{P(\text{cat} \mid \text{The})}_{\text{one word of context}} \times \underbrace{P(\text{sat} \mid \text{The cat})}_{\text{two words of context}}$$

In general, for a sequence of T tokens $x_1, \ldots, x_T$:

$$p_\theta(x_1, \ldots, x_T) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t})$$

In words: multiply together the probability of each token given everything before it. ($x_{<t}$ is everything before position t; θ is the model's weights.)

Why this matters

Splitting a sentence up like this is just the chain rule of probability, so no approximation has been introduced. It also turns a sequence of T tokens into T next-token training examples.

The training objective

Training adjusts the weights so that **real text gets high probability**. Written as something to minimise:

$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})$$

In words: at every position, look at the probability the model gave the word that actually came next, and penalise it for being small.

Why the logarithm

It turns the product on the previous slide into a sum, which is numerically stable and gives a per-token average that does not depend on sentence length.

What it is used for

Cross-entropy loss provides the training signal used to update the model's weights. It is the same loss used in classification, with the vocabulary entries acting as classes.

Take away

The same basic objective is used to train small classroom models and large production models.

From counts to learned representations

One way to estimate these probabilities is to **count**: take a large corpus and count how often "sat" follows "The cat". This gives an **n-gram model**. N-grams remained strong baselines into the early 2010s as neural language models increasingly replaced them.

Where counting struggles

Most contexts you meet at test time were **never seen in training**, so the count is zero and the probability is zero.

And counts cannot share what they learn. "The cat sat" tells the model nothing about "The dog sat", because the two contexts are just different strings.

What vectors fix

Each word becomes a **vector**, and similar words get similar vectors, so "dog" inherits statistics from "cat" for free.

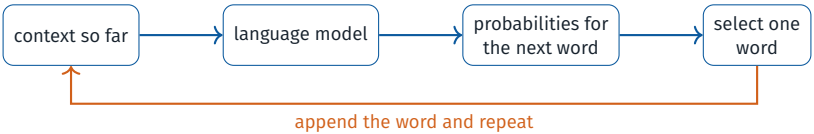
This sharing lets the model assign sensible scores to contexts that never appeared verbatim in the training data. This ability to generalise beyond exact matches is why learned representations matter.

The lineage: n-gram → neural LM (Bengio 2003) → RNN/LSTM → **attention**, invented to fix the RNN bottleneck in translation → **transformer**, which kept attention and removed the recurrence.

Source: Bengio et al., *A Neural Probabilistic Language Model*, 2003; Vaswani et al., *Attention Is All You Need*, 2017.

From probabilities to text: where "generative" comes in

At each step, the model assigns a probability to every possible next word. These probabilities do not yet produce text: a **selection rule** must choose one word.



Most likely (*greedy*). Always select the word with the highest probability. This is predictable but can become repetitive.

Sampling. Choose randomly according to the probabilities: likely words are chosen more often. This adds variety, so the same prompt can continue differently.

Take away

Generation repeatedly predicts probabilities, selects one word, and adds it to the context. Greedy decoding and sampling are two ways to select.

Perplexity

Cross-entropy is useful for training, but its scale is hard to interpret. **Perplexity** turns it into an approximate "number of plausible choices":

$$\mathrm{PPL} = e^{\mathcal{L}}$$

For example, if the average loss is $\mathcal{L} = 2$, then $\mathrm{PPL} = e^2 \approx 7.4$. The model is about as uncertain as a uniform choice among seven or eight tokens at each step.

Lower is better. Perfect prediction gives $\mathrm{PPL} = 1$.

What it is used for: tracking progress during training and comparing checkpoints on the same tokenized validation data.

Compare perplexity only with the same tokenizer

Changing the tokenizer changes what counts as one prediction. A larger vocabulary produces fewer but harder tokens, so its perplexity is on a different scale. Use *bits per byte* for comparisons across tokenizers.

What next-token prediction can—and cannot—teach

Why useful abilities emerge

To predict text well, a model must learn recurring patterns in language: grammar, meaning, facts, discourse, code and common forms of reasoning.

The better it represents these patterns, the better it can predict what comes next.

What the objective does not guarantee

Frequent text is not necessarily true or useful. A fluent continuation is therefore not evidence that a claim is correct.

A simple example

"The Helsinki train was delayed, so Anna arrived" **late**

Predicting "late" requires more than the previous word. The model needs to connect the delay, the train and Anna across the sentence.

Repeating this task over a large corpus encourages the model to learn such relationships.

A base model learns to *continue text*; it is not automatically trained to follow a user's instructions. That is the topic of week 2.

What comes with a model release?

- **weights** — the `.safetensors` files
- **tokenizer** — vocabulary plus merge rules
- **chat template** — a Jinja string that turns a conversation into one flat sequence
- **generation config** — default sampling parameters
- **inference code** — the architecture itself

Gotcha

Three of those five are **not** in the weights file, and each of them can silently ruin your evaluation.

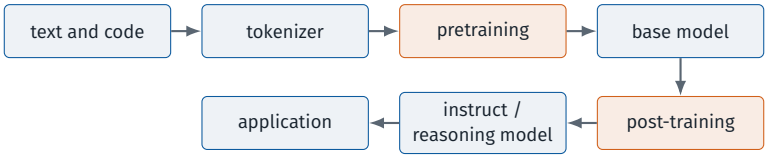
The most common failure in Lab 2: applying a chat template to a **base** model, which has never seen those tokens.

Browse open-weight model releases at `huggingface.co/models`.

Take away

Model behaviour depends on the weights and on how text is tokenized, formatted and decoded.

The life cycle of a language model



Pretraining
Learn broad patterns from a large corpus by predicting the next token.

Post-training
Teach the base model to follow instructions, use tools or reason in a desired format.

Application
Add prompting, retrieval, tools, evaluation and an inference system.

The same model can play different roles

Base model

Continues text in the style of its training data.

Useful for studying pretraining and as a starting point for further training, but not designed as a chat assistant.

Instruct model

Post-trained on instructions and preferences. Expects a particular chat template and usually follows user requests more directly.

Reasoning model

Trained to spend additional tokens on difficult problems before producing an answer.

Model inside a system

Retrieval can supply documents; tools can supply calculations or external actions. The model remains the text-prediction component.

Week 2 covers post-training; weeks 5 and 6 add retrieval and tools.

Tokenization

Characters, words or subwords?

	Advantage	Trade-off
characters	tiny vocabulary, no OOV	sequences 4–5$\times$ longer; attention is $O(n^2)$
words	short sequences, interpretable	open vocabulary; every typo, compound and inflection is OOV
subwords	bounded vocabulary, no OOV, reasonable length	the segmentation depends on the training data

Byte-level BPE takes the subword route and removes the last OOV case: the base alphabet is the **256 byte values**, so text in any language, including emoji, can be represented through its bytes.

Byte-level BPE, worked

Start with individual bytes. Find a frequent adjacent pair, merge it into a new token, and record the merge. Repeat until the vocabulary is large enough.

Step	Merge learned	A word after the merge
0	none yet	`l o w e s t`
1	`l` + `o` → `lo`	`lo w e s t`
2	`lo` + `w` → `low`	`low e s t`
3	`e` + `s` → `es`	`low es t`
4	`es` + `t` → `est`	`low est`

This is one possible merge sequence learned from a tiny corpus containing words such as `low`, `lower`, `newest` and `widest`. A real tokenizer learns thousands of merges.

Why it matters: these merges determine the units the model learns to represent and how many tokens each input consumes.

Choosing the vocabulary size

What a larger vocabulary buys

- common words and word parts fit into fewer tokens
- sequences become shorter
- more text fits into the same context window
- attention has fewer positions to process

What it costs

Every vocabulary item needs an input embedding and an output score. A larger vocabulary therefore uses more model parameters.

Rare tokens also receive fewer training examples than reusable subword pieces.

Decoder-only examples—not target values

GPT-2 (English)	Poro (Finnish + English)	Qwen2.5 (multilingual)	Gemma 2 (multilingual)
50.3k	**128k**	**151.9k**	**256k**

Source: Model tokenizer/config files; Luukkonen et al. 2024 (Poro).

Fertility: why some languages use more tokens

Fertility = tokens per word. The same sentence, the same tokenizer, different languages:

Language	Typical fertility	Why
English	$\approx$ 1.2–1.3	dominates the training corpus; common words are single tokens
German	$\approx$ 1.6–2.0	compounding
Finnish	$\approx$ 2.0–2.8	many endings create many word forms; less Finnish in the training data
Thai, Georgian, Amharic	3–5+	little training data, non-Latin script

talossanikinko $\rightarrow$ a single Finnish word; several tokens in every mainstream tokenizer.

Lab 1, Part A: measure this yourself on FLORES-200, for your own tokenizer and for Qwen3.5's and Gemma 4's.

What high fertility costs you

Cost

APIs bill per token. The same content may cost more to process in Finnish than in English.

Context

A fixed token window can hold substantially less Finnish text than English text.

Quality

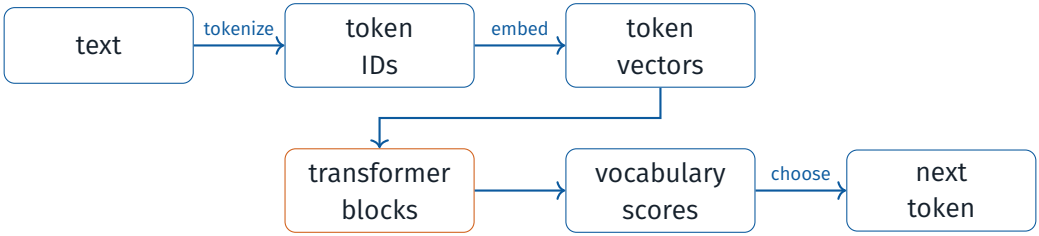
More tokens per unit of meaning can make dependencies longer and spread the available training signal across more tokens.

The tokenizer is usually fixed before pretraining

The tokenizer is trained first and normally kept fixed for the rest of training. Fine-tuning cannot fully compensate for poor segmentation.

How transformers process context

A TRANSFORMER LANGUAGE MODEL, END TO END



Each transformer block alternates between:

Attention

Lets each token gather relevant information from other positions.

Feed-forward network

Transforms the information at each position independently.

Stack the block many times, then map the final vector to one score for every vocabulary item.

Why transformers use attention

Language depends on context. In *"The animal did not cross the road because it was tired"*, understanding *it* requires information from several words earlier.

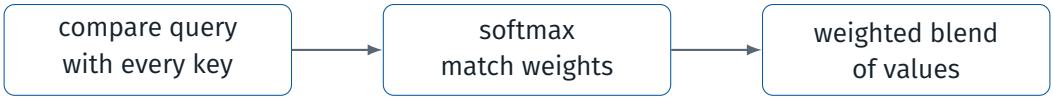
Earlier recurrent models	Self-attention
An RNN carries context forward through one hidden state, one token at a time.	Each position can retrieve information directly from relevant earlier positions.
Information travels through many sequential steps, and the positions cannot be computed in parallel during training.	During training, all positions can be computed in parallel; a causal mask prevents access to future tokens.

Inside a transformer block, **attention** exchanges information between positions; the **feed-forward network** transforms each position independently.

Queries, keys and values

Think of attention as a learned search over the context. Each token produces:

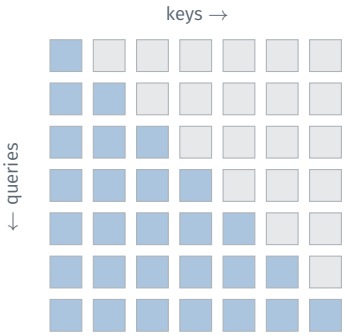
Query	Key	Value
What information am I looking for?	What information do I contain?	What information should I pass on?



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

What it is used for: letting each token retrieve the parts of the context that are most useful for its current representation.

The causal mask



Each row shows which earlier tokens one position is allowed to use.

The grey upper triangle is the **causal mask**: future positions are hidden.

Why it is used: during training, we can compute all positions **in parallel** without letting a token see the answer it should predict.

During generation, each new token adds one row. This sequential step is one reason generation is slower than training.

Multiple heads

Multiple heads let the model retrieve several kinds of contextual information at once. Run h attentions in parallel with *different* learned projections, each of width $d_{\text{head}} = d_{\text{model}}/h$, then concatenate and project.

- Same cost as one wide head, but h separate ways to route information
- Empirically heads specialise: local dependencies, syntactic relations, long-range semantic links, positional patterns
- Many heads turn out to be prunable after training — a hint that this is redundancy as much as specialisation

The inference-time cost

Every head keeps its own K and V for every token in the context, for every layer, for every request in the batch. That stored state is the **KV cache**.

Why attention becomes expensive

Compute: compare token pairs

For n tokens, $QK^\top$ contains n^2 scores.

Double the context length and attention performs roughly **four times as much** pairwise work.

Memory: keep the past

During generation, the model stores earlier keys and values so it does not recompute them at every step.

This **KV cache** grows with the context length and the number of simultaneous requests.

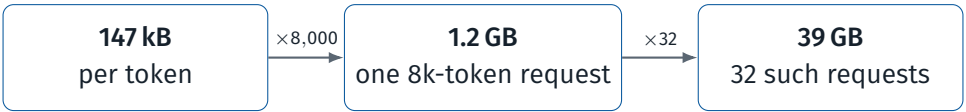
Take away

Compute limits how much context one request can process. KV-cache memory often limits how many requests a server can handle at once.

KV CACHE: A WORKED EXAMPLE

For a 4B-class model with 36 layers, 8 KV heads, head width 128 and bf16 values:

$$\underbrace{36}_{\text{layers}} \times \underbrace{8 \times 128}_{\text{width}} \times \underbrace{2}_{\text{keys } \textit{and} \text{ values}} \times \underbrace{2}_{\text{bytes}} \approx 147\,\text{kB}$$ **per token**



The model's bf16 weights take about **8 GB**; in this example, the cache takes much more. This is why serving systems spend so much effort on KV-cache management.

The model landscape

Useful distinctions between models

Axis	What it means in practice
dense vs MoE	dense activates all parameters; MoE routes each token through a subset of expert layers
open-weight vs open-source	releasing *weights* does not necessarily mean releasing the data, code and full training recipe
licence	Apache 2.0 / MIT are permissive; other licences may restrict particular uses
base vs instruct	base continues text; instruct follows requests through a chat template. Week 2 compares both
reasoning vs not	uses extra generation for intermediate reasoning; this can help on some difficult tasks, but adds latency and does not help uniformly
decoder vs encoder	decoder models generate text; encoder models remain common for *embeddings* and retrieval — week 5

How to read a new model release

Use this checklist for your project and again in week 7.

1. Start with `config.json`, then read the release post
2. Which variants exist: base, instruct, reasoning?
3. How large is it, and what hardware does it require?
4. What is its context length?
5. Which tokenizer and chat template does it use?
6. How well does the tokenizer cover *your* language?
7. What does the licence allow?
8. What data and post-training recipe are documented?
9. Finally, inspect benchmarks and their evaluation setup

Gotcha

A model card is a starting point, not independent evidence. Test the model on data that represents your own task.

Roihu in ten minutes

Getting access

Before the first lab

1. Create a CSC account
2. Join the invited course project
3. Accept the Roihu terms

Choose how to connect

Browser

Open `www.roihu.csc.fi`. JupyterLab provides a file browser, optional notebooks and a terminal. MFA is required.

SSH from your own terminal

Add and sign an SSH key in MyCSC (certificate valid for 24 hours):

```
ssh USERNAME@roihu-gpu.csc.fi
```

Course repository (not cloned automatically)

```
git clone https://github.com/Helsinki-NLP/LLM-Course-2026.git
```

Register today: account approval can take time.

Two ways to run

Mode	Partition	Limits	Used for
Interactive CPU	`interactive`	36 h, up to 32 cores	tokenizers, data prep, writing code
Interactive GPU	`gpuinteractive`	12 h, one GH200	lab sessions, prompting, LoRA SFT, vLLM
Batch, short	`gputest`	15 min	**debugging your script**
Batch, long	`gpumedium`	36 h, 1–4 GPUs	tiny-GPT pretraining, GRPO, long fine-tunes

The GPU nodes use ARM CPUs

The GPU nodes run **ARM**. Build any virtual environment on Roihu's ARM login nodes, and check that any additional package supports the platform before depending on it.

A BATCH JOB, LINE BY LINE

```
#!/bin/bash
#SBATCH --account=project_2021893   # the course project
#SBATCH --partition=gputest         # <- gputest first, always
#SBATCH --nodes=1
#SBATCH --gres=gpu:gh200:1          # one GH200
#SBATCH --cpus-per-task=16
#SBATCH --mem=64G
#SBATCH --time=00:15:00             # gputest caps at 15 minutes
#SBATCH --output=logs/%x-%j.out     # job name and job id in the filename

module purge
module load python-pytorch/2.13     # the course environment

srun python train_tiny_gpt.py --config configs/small.yaml
```

`sbatch job.sh` to submit `squeue --me` to watch `scancel <jobid>` to stop `seff <jobid>` to see what you actually used

Lab 1 — take-home work

A. Tokenizer *CPU or compute node*

Train a byte-level BPE on Finnish Wikipedia at 2–3 vocabulary sizes, then compare fertility against Qwen3.5 and Gemma 4.

B. Tiny GPT *GPU batch*

Train a 10–30M-parameter decoder with your tokenizer. Script and Slurm template provided; about 20 min per run.

C. Scaling

Three model sizes at a fixed token budget. Plot validation loss against compute.

D. What scale buys

Run the same few-shot prompt on your tiny model and on a larger pretrained base model.

E. Project

Choose a task, model and held-out set.

A strong report can include

a fertility table • loss curves • a scaling plot • one paragraph comparing your run's compute with a published frontier run

There is no lab session this week

Thursday is a lecture, so start early and use the course channel.

Follow `Week-1/Lab-1/README.md` in the course repository. The scaling plot is explained on Thursday.

Lab 1: acceptable work and strong work

Minimum acceptable

- run the smoke test
- compare Finnish and English with at least two vocabulary sizes
- train one supplied tiny-GPT configuration
- plot training and validation loss
- record parameters, tokens, runtime and estimated compute
- explain whether it learned, your evidence and the limitations

Strong work

- compares three languages, three vocabulary sizes and published model tokenizers
- compares three model sizes with the same tokenizer, token count and random seed
- includes a clearly labelled scaling plot
- compares the tiny model with a pretrained base model
- investigates unexpected results and documents how to reproduce the experiment

Take away

There is no required loss threshold. Clear evidence, interpretation and reproducibility matter more than obtaining a particular result.

Three things to remember

Three things to remember

1. **Language models predict distributions over the next token.** Generating text means choosing a token, adding it to the context and repeating.
2. **Tokenization and architecture shape both quality and cost.** Transformers build contextual token representations with attention and feed-forward layers.
3. **An LLM application is more than model weights.** Post-training, prompting, retrieval, tools and evaluation all affect what users see.

Thursday: where the weights come from. Data pipelines, scaling laws, and what a real pretraining run looks like.

Tonight: create your CSC account. You may also start considering a task and model for the later labs.

Reading

Core

- Stanford CS336, *Language Modeling from Scratch*
- Karpathy, *nanoGPT* and *nanochat*
- Vaswani et al. 2017, *Attention Is All You Need*

Architecture, current

- Raschka, *A Visual Guide to Attention Variants in Modern LLMs* (Mar 2026)
- Raschka, *Recent Developments in LLM Architectures* (May 2026)
- Raschka, *A Dream of Spring for Open-Weight LLMs* (Feb 2026)

Specific mechanisms

- Ainslie et al. 2023 — GQA
- DeepSeek-V2 / V3 technical reports — MLA, MoE
- Raschka, *From DeepSeek V3 to V3.2* (Dec 2025) — DSA
- Su et al. 2021 — RoPE
- Shazeer 2020 — GLU variants

Roihu

- `docs.csc.fi/computing/systems-roihu`
- `docs.csc.fi/computing/webinterface/jupyter-for-courses`